Name: Darshika Singh
Roll No: 2401114
Batch: T22

Experiment 3: To deploy multi-service applications using Docker Compose

Aim: To deploy multi-service applications using Docker Compose

LO Mapped: LO4

LO: Containerize, orchestrate and deploy applications using Docker, Docker Compose, Kubernetes and cloud infrastructure tools.

Theory:

1. What is Docker Compose*?*

Docker Compose is an open-source orchestration tool designed for defining, configuring, and running multi-container Docker applications.

While the standard Docker CLI requires executing individual docker run commands for every container—manually specifying ports, environment variables, network links, and volume mounts—Docker Compose allows developers to define an entire application stack in a single, declarative configuration file named **docker-compose.yml**.

Using this single source of truth, you can launch, stop, and manage all interconnected services using a single command line call (docker compose up).

Key Concepts in Docker Compose

Concept	Description
Service	An isolated containerized component of your application (e.g., a MySQL database, an Nginx web server, or a Redis cache). A single service definition can also be scaled across multiple container instances.
Volume	A persistent storage mechanism managed by Docker that allows data (like database records) to survive container restarts and destructions.
Network	An isolated virtual communication bridge automatically created by Docker Compose so containers within the same stack can talk to each other using their service names as domain names.
Environment	Configuration key-value pairs (passwords, hostnames, ports) passed into containers at runtime to alter behavior without modifying container images.

Why Use Docker Compose?

1. **Declarative Architecture:** Instead of remembering complex CLI commands, all configurations are stored in source-controlled code (docker-compose.yml).
2. **Automated Networking:** Services defined within the same docker-compose.yml file are automatically connected to a shared internal network. Containers can reference each other directly by service name (e.g., adminer connects to MySQL via host name db instead of an IP address).
3. **Environment Isolation:** Compose creates isolated project environments, preventing port conflicts and namespace collisions on the host machine.
4. **Single-Command Lifecycle Management:** Spin up (docker compose up), inspect (docker compose ps), and destroy (docker compose down) complex multi-tier applications effortlessly.

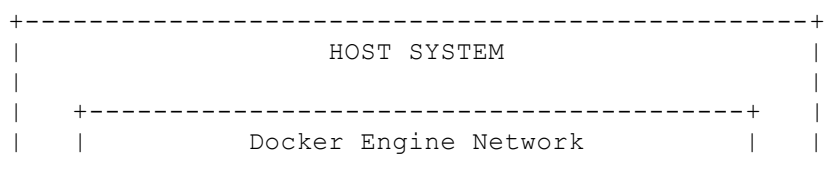
Anatomy of a docker-compose.yml File

The YAML file uses strict two-space indentation and key directive keywords:

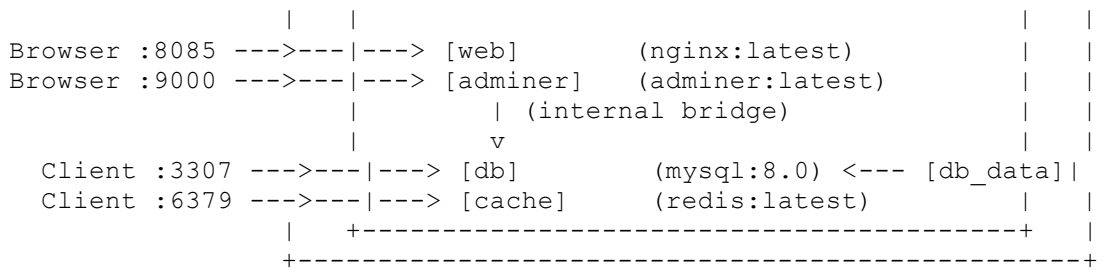
- version / services: Top-level root tag declaring all individual container configurations.
- image: Specifies the pre-built Docker image to pull from Docker Hub.
- container_name: Assigns a custom readable name to the running container instance.
- ports: Maps **"Host_Port:Container_Port"**. Traffic arriving at the host machine's port is forwarded to the container's internal port.
- environment: Sets internal environment variables (such as database passwords and default database names).
- volumes: Mounts host paths or named Docker volumes to internal container directories.
- depends_on: Establishes startup dependency order (e.g., ensure db starts before adminer).

2. Multi-Service Application Architecture

For this experiment, we orchestrate a **4-service stack** consisting of a relational database, a database administration client, a web proxy/server, and an in-memory key-value cache.



Name: Darshika Singh
Roll No: 2401114
Batch: T22



Services Breakdown

1. **db (MySQL 8.0):** Relational database storing persistent tables inside a named volume (db_data). Bound to host port 3307.
2. **adminer (Adminer):** Lightweight database management web interface listening on host port 9000, configured to manage the db service.
3. **web (Nginx):** High-performance web server listening on host port 8085.
4. **cache (Redis):** In-memory data structure store listening on host port 6379 for fast key-value caching.

3. Step-by-Step Implementation & Execution

Step 1: Create Project Directory and Configuration File: File Setup.

Create a new project directory on your terminal and initialize the docker-compose.yml manifest:

Bash

```
mkdir docker_lab_exp3

cd docker_lab_exp3

nano docker-compose.yml
```

Paste the following corrected deployment file:

YAML

```
services:

 # Service 1: Relational Database

 db:
```

```
  image: mysql:8.0
  container_name: lab_mysql_db
  environment:
    MYSQL_ROOT_PASSWORD: rootpassword
    MYSQL_DATABASE: lab_database
  ports:
    - "3307:3306"
  volumes:
    - db_data:/var/lib/mysql

# Service 2: Web Management Interface
adminer:
  image: adminer
  container_name: lab_web_admin
  restart: always
  ports:
    - "9000:8080"
  depends_on:
    - db

# Service 3: Web Server
```

```
 web:
  image: nginx:latest
  container_name: web
  ports:
   - "8085:80"


 # Service 4: In-Memory Cache
 cache:
  image: redis:latest
  container_name: redis
  ports:
   - "6379:6379"


volumes:
 db_data:
```

Step 2: Build and Launch the Application Stack: Command: docker compose up -d.

Execute the build command with the -d (detached mode) flag to download all required images from Docker Hub, create virtual networks, allocate named volumes, and start all 4 containers in the background:

Bash

```
docker compose up -d
```

Note: The terminal will display image pulling progress followed by container creation notices for lab_mysql_db, lab_web_admin, web, and redis.

Name: Darshika Singh
Roll No: 2401114
Batch: T22

Step 3: Verify Container Deployment and Status: Command: docker compose ps.

Confirm that all 4 containers are up, running, and mapped to their designated ports:

Bash

```
docker compose ps
```

You can also run standard docker diagnostics:

Bash

```
docker ps --format "table {{.Names}}\t{{.Status}}\t{{.Ports}}"
```

Step 4: Access Web-Based Services via Browser:HTTP Visual Verification.

Open your browser to test the active web interfaces:

1. **Nginx Web Server:** Navigate to http://localhost:8085.
2. **Adminer Database GUI:** Navigate to http://localhost:9000. Log in using:
    - **System:** MySQL
    - **Server:** db
    - **Username:** root
    - **Password:** rootpassword
    - **Database:** lab_database

Step 5: Verify CLI Services (MySQL & Redis): Interactive Execution via Shell.

Execute interactive container sessions to test non-HTTP endpoints directly:

- **Testing MySQL Database:**

Bash

```
docker exec -it lab_mysql_db mysql -u root -prootpassword -e "SHOW DATABASES;"
```

- **Testing Redis Cache:**

Bash

```
docker exec -it redis redis-cli ping
```

Name: Darshika Singh
Roll No: 2401114
Batch: T22

(Expected output response: PONG)

Step 6: Stack Teardown and Cleanup: Lifecycle Management.

When experimentation is complete, stop and remove all services, networks, and resources cleanly:

Bash: docker compose down

4. Lab Evidence & Screenshots Checklist

Attach your actual execution screenshots under each designated section below:

Screenshot 1: Directory Setup & docker-compose.yml File

```
services:
  # Service 1: The Relational Database
  db:
    image: mysql:8.0
    container_name: lab_mysql_db
    environment:
      MYSQL_ROOT_PASSWORD: rootpassword
      MYSQL_DATABASE: lab_database
    ports:
      - "3307:3306"
    volumes:
      - db_data:/var/lib/mysql

  # Service 2: The Web Interface (Adminer)
  adminer:
    image: adminer
    container_name: lab_web_admin
    restart: always
    ports:
      - "9000:8080"
    depends_on:
      - db

  # Service 3: Web Server
```

Name: Darshika Singh
Roll No: 2401114
Batch: T22

```
  web:
    image: nginx:latest
    container_name: web
    ports:
      - "8085:80"

  # Service 4: Cache
  cache:
    image: redis:latest
    container_name: redis
    ports:
      - "6379:6379"

# Persistent volumes for storage
volumes:
  db_data:
```

Screenshot 2: Launching Stack (docker compose up -d)

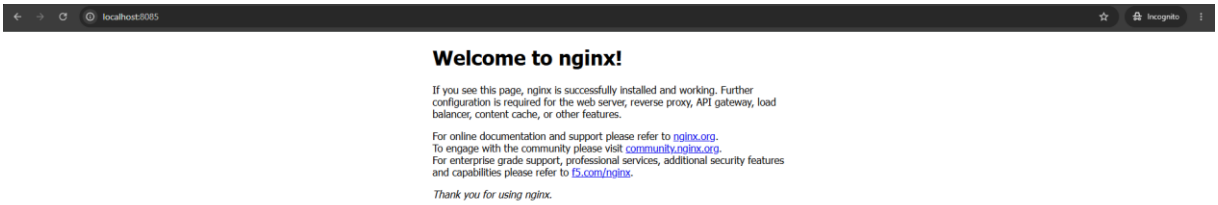
```
C:\Users\L15\compose_experiment>docker compose up -d
[+] up 4/4
 ✔Container redis          Running
 ✔Container lab_mysql_db   Running
 ✔Container web            Running
 ✔Container lab_web_admin  Running
```

Screenshot 3: Active Container Listing (docker compose ps)

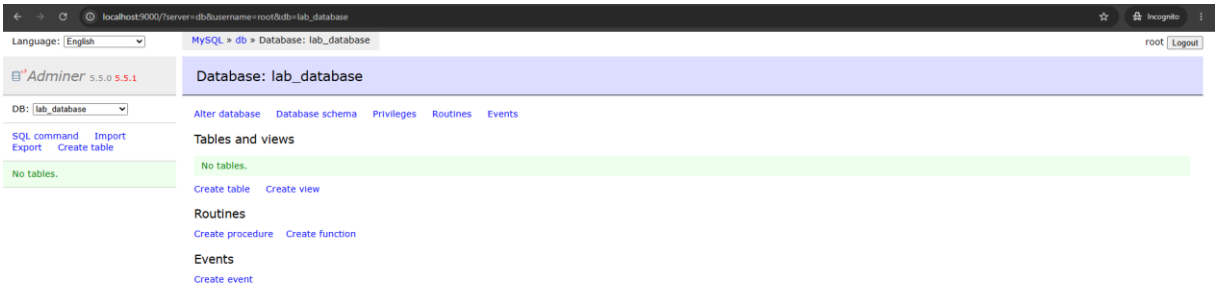
```
C:\Users\L15\compose_experiment>docker compose ps
NAME            IMAGE          COMMAND                  SERVICE   CREATED              STATUS          PORTS
lab_mysql_db    mysql:8.0      "docker-entrypoint.s…"   db        Less than a second ago   Up 55 minutes   0.0.0.0:3307->3306/tcp, [::]:3307->3306/tcp
lab_web_admin   adminer        "entrypoint.sh docke…"   adminer   Less than a second ago   Up 52 minutes   0.0.0.0:9000->8080/tcp, [::]:9000->8080/tcp
redis           redis:latest   "docker-entrypoint.s…"   cache     Less than a second ago   Up 31 minutes   0.0.0.0:6379->6379/tcp, [::]:6379->6379/tcp
web             nginx:latest   "/docker-entrypoint.…"   web       Less than a second ago   Up 31 minutes   0.0.0.0:8085->80/tcp, [::]:8085->80/tcp
```

Screenshot 4: Nginx Web Server Interface

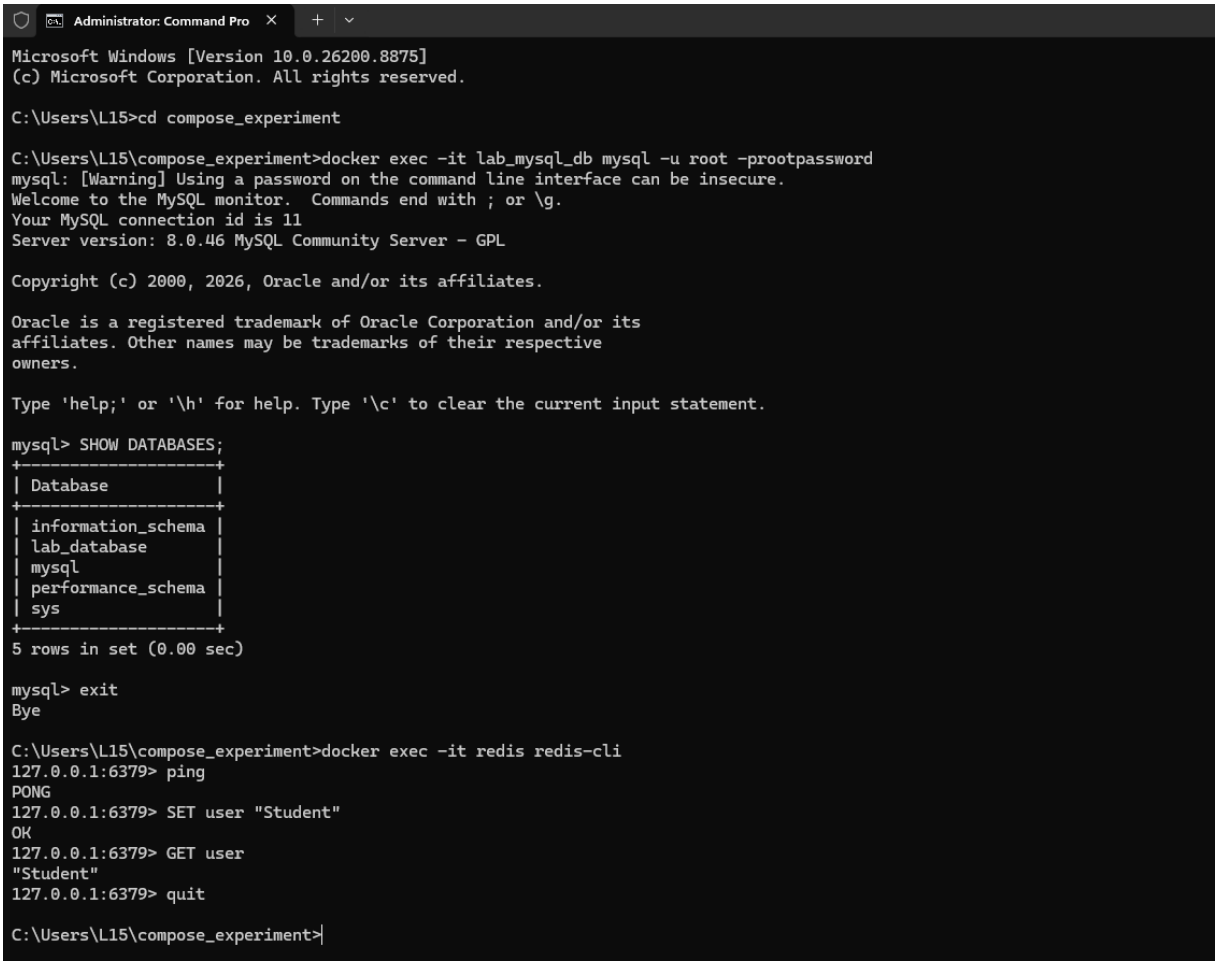
Name: Darshika Singh
Roll No: 2401114
Batch: T22



Screenshot 5: Adminer Database Management GUI

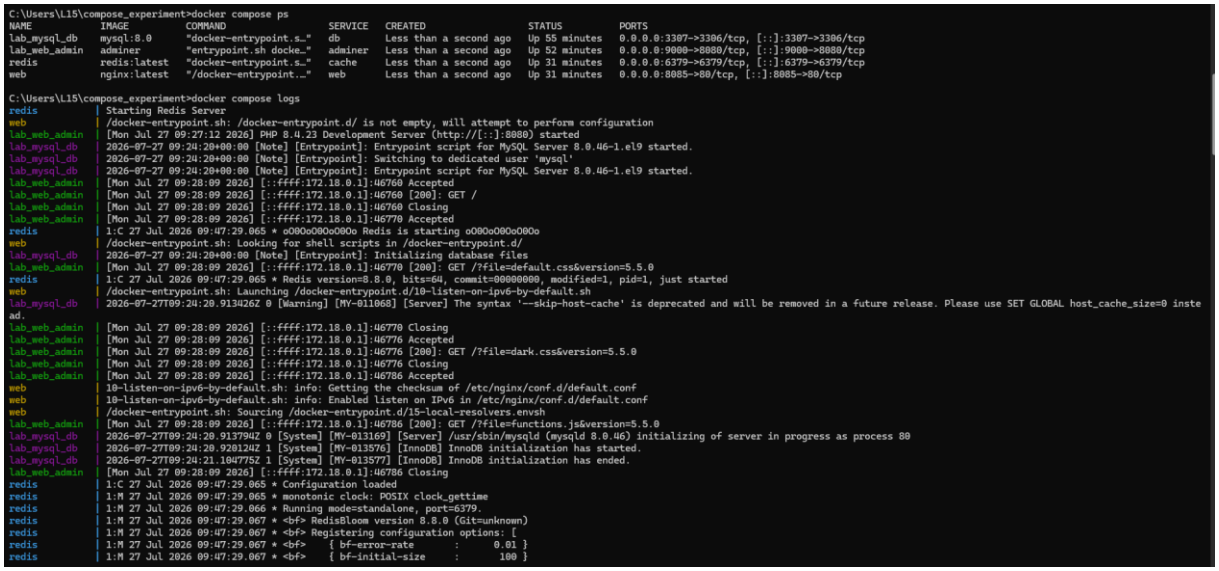


Screenshot 6: Redis and MySQL CLI Tests



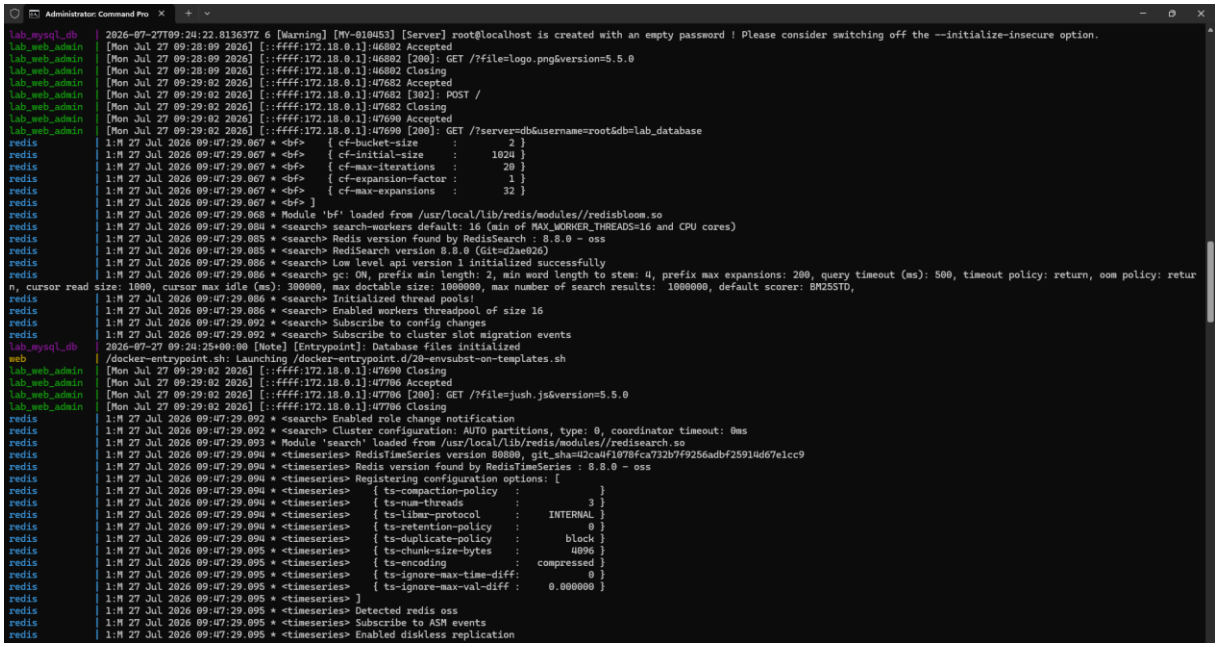
Name: Darshika Singh
Roll No: 2401114
Batch: T22

Screenshot 7: Rest all commands executed (Given by mam)



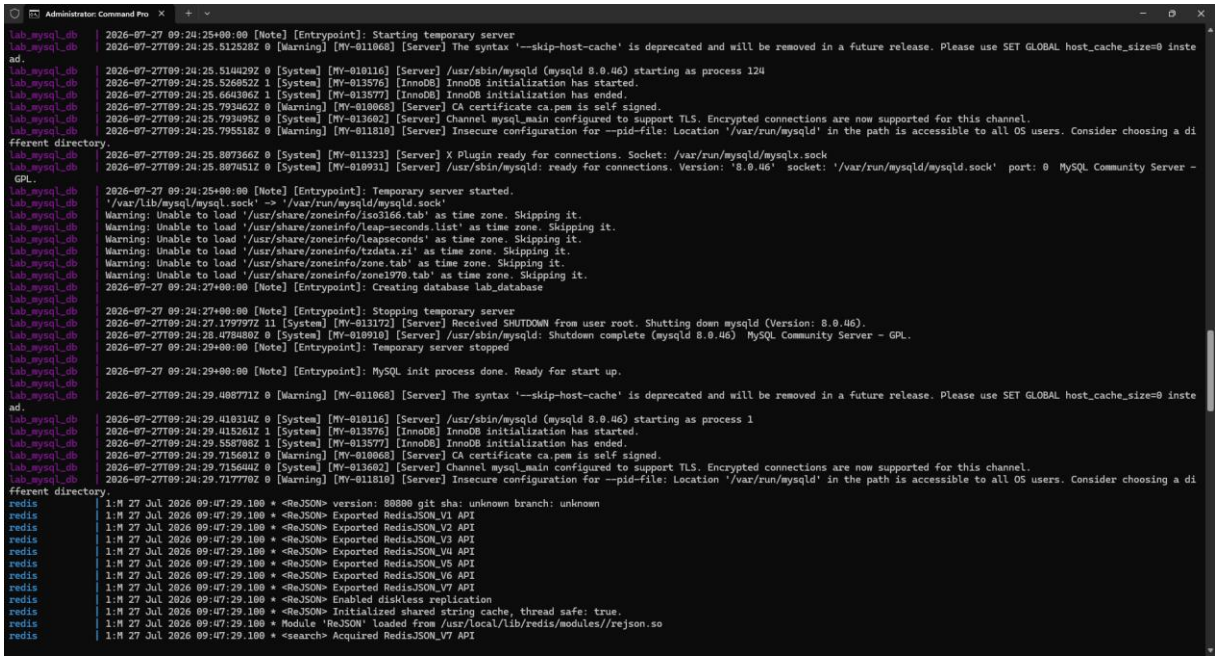
```
C:\Users\L15\compose_experiment>docker compose ps
NAME            IMAGE          COMMAND                  SERVICE   CREATED                  STATUS          PORTS
lab_mysql_db    mysql:8.0      "docker-entrypoint.s…"   db        Less than a second ago   Up 55 minutes   0.0.0.0:3307->3306/tcp, [::]:3307->3306/tcp
lab_web_admin   adminer        "entrypoint.sh docke…"   adminer   Less than a second ago   Up 52 minutes   0.0.0.0:9000->8080/tcp, [::]:9000->8080/tcp
redis           redis:latest   "docker-entrypoint.s…"   cache     Less than a second ago   Up 31 minutes   0.0.0.0:6379->6379/tcp, [::]:6379->6379/tcp
web             nginx:latest   "/docker-entrypoint.…"   web       Less than a second ago   Up 31 minutes   0.0.0.0:8085->80/tcp, [::]:8085->80/tcp

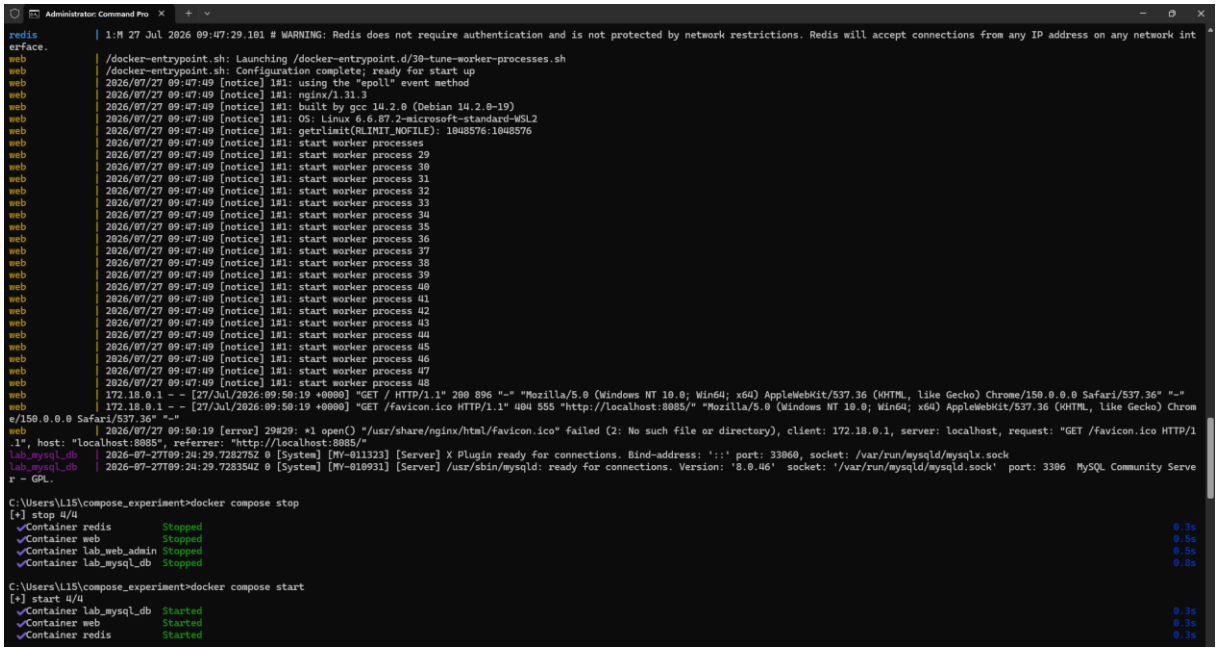
C:\Users\L15\compose_experiment>docker compose logs
redis           | Starting Redis Server
web             | /docker-entrypoint.sh: /docker-entrypoint.d/ is not empty, will attempt to perform configuration
lab_web_admin   | [Mon Jul 27 09:27:12 2026] PHP 8.4.23 Development Server (http://[::]:8080) started
lab_mysql_db    | 2026-07-27 09:24:20+00:00 [Note] [Entrypoint]: Entrypoint script for MySQL Server 8.0.46-1.el9 started.
lab_mysql_db    | 2026-07-27 09:24:20+00:00 [Note] [Entrypoint]: Switching to dedicated user 'mysql'
lab_mysql_db    | 2026-07-27 09:24:20+00:00 [Note] [Entrypoint]: Entrypoint script for MySQL Server 8.0.46-1.el9 started.
lab_web_admin   | [Mon Jul 27 09:28:09 2026] [::ffff:172.18.0.1]:46760 Accepted
lab_web_admin   | [Mon Jul 27 09:28:09 2026] [::ffff:172.18.0.1]:46760 [200]: GET /
lab_web_admin   | [Mon Jul 27 09:28:09 2026] [::ffff:172.18.0.1]:46760 Closing
lab_web_admin   | [Mon Jul 27 09:28:09 2026] [::ffff:172.18.0.1]:46770 Accepted
redis           | 1:C 27 Jul 2026 09:47:29.065 * oO0OoO0OoO0Oo Redis is starting oO0OoO0OoO0Oo
web             | /docker-entrypoint.sh: Looking for shell scripts in /docker-entrypoint.d/
lab_mysql_db    | 2026-07-27 09:24:20+00:00 [Note] [Entrypoint]: Initializing database files
lab_web_admin   | [Mon Jul 27 09:28:09 2026] [::ffff:172.18.0.1]:46770 [200]: GET /?file=default.css&version=5.5.0
redis           | 1:C 27 Jul 2026 09:47:29.065 * Redis version=8.8.0, bits=64, commit=00000000, modified=1, pid=1, just started
web             | /docker-entrypoint.sh: Launching /docker-entrypoint.d/10-listen-on-ipv6-by-default.sh
lab_mysql_db    | 2026-07-27T09:24:20.913426Z 0 [Warning] [MY-011068] [Server] The syntax '--skip-host-cache' is deprecated and will be removed in a future release. Please use SET GLOBAL host_cache_size=0 instead.
lab_web_admin   | [Mon Jul 27 09:28:09 2026] [::ffff:172.18.0.1]:46770 Closing
lab_web_admin   | [Mon Jul 27 09:28:09 2026] [::ffff:172.18.0.1]:46776 Accepted
lab_web_admin   | [Mon Jul 27 09:28:09 2026] [::ffff:172.18.0.1]:46776 [200]: GET /?file=dark.css&version=5.5.0
lab_web_admin   | [Mon Jul 27 09:28:09 2026] [::ffff:172.18.0.1]:46776 Closing
lab_web_admin   | [Mon Jul 27 09:28:09 2026] [::ffff:172.18.0.1]:46786 Accepted
web             | 10-listen-on-ipv6-by-default.sh: info: Getting the checksum of /etc/nginx/conf.d/default.conf
web             | 10-listen-on-ipv6-by-default.sh: info: Enabled listen on IPv6 in /etc/nginx/conf.d/default.conf
web             | /docker-entrypoint.sh: Sourcing /docker-entrypoint.d/15-local-resolvers.envsh
lab_web_admin   | [Mon Jul 27 09:28:09 2026] [::ffff:172.18.0.1]:46786 [200]: GET /?file=functions.js&version=5.5.0
lab_mysql_db    | 2026-07-27T09:24:20.913794Z 0 [System] [MY-013169] [Server] /usr/sbin/mysqld (mysqld 8.0.46) initializing of server in progress as process 80
lab_mysql_db    | 2026-07-27T09:24:20.920124Z 1 [System] [MY-013576] [InnoDB] InnoDB initialization has started.
lab_mysql_db    | 2026-07-27T09:24:21.104775Z 1 [System] [MY-013577] [InnoDB] InnoDB initialization has ended.
lab_web_admin   | [Mon Jul 27 09:28:09 2026] [::ffff:172.18.0.1]:46786 Closing
redis           | 1:C 27 Jul 2026 09:47:29.065 * Configuration loaded
redis           | 1:M 27 Jul 2026 09:47:29.065 * monotonic clock: POSIX clock_gettime
redis           | 1:M 27 Jul 2026 09:47:29.066 * Running mode=standalone, port=6379.
redis           | 1:M 27 Jul 2026 09:47:29.067 * <bf> RedisBloom version 8.8.0 (Git=unknown)
redis           | 1:M 27 Jul 2026 09:47:29.067 * <bf> Registering configuration options: [
redis           | 1:M 27 Jul 2026 09:47:29.067 * <bf>     { bf-error-rate       :      0.01 }
redis           | 1:M 27 Jul 2026 09:47:29.067 * <bf>     { bf-initial-size     :       100 }
```



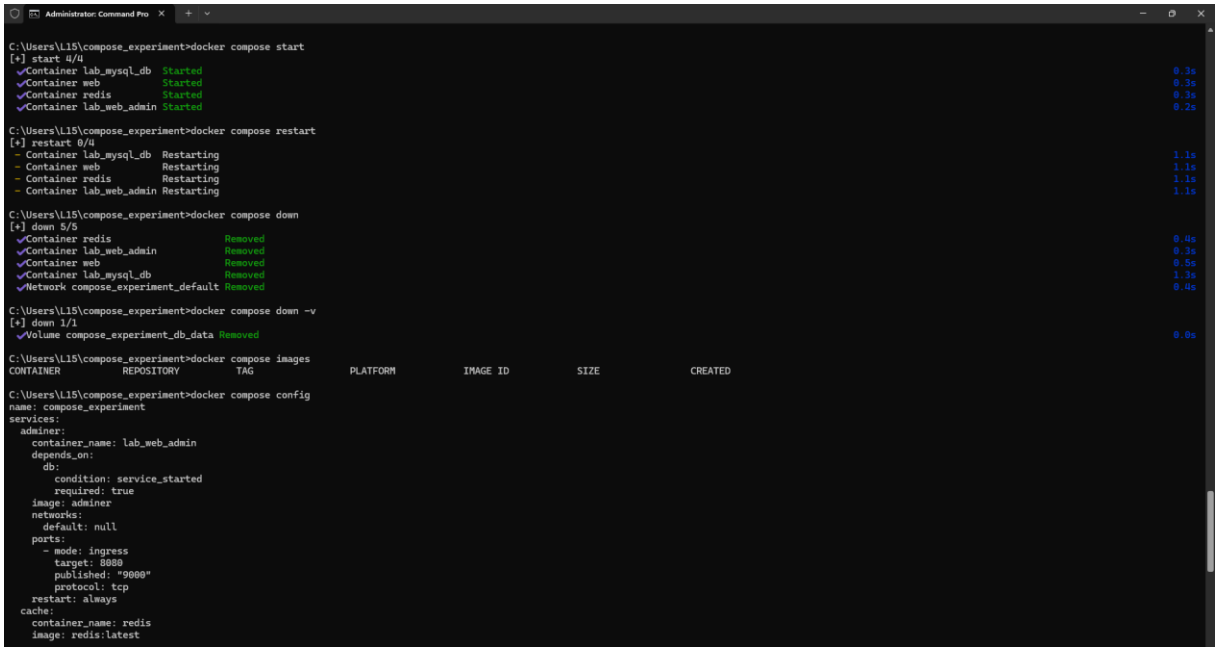
```
Administrator: Command Pro
lab_mysql_db    | 2026-07-27T09:24:22.813637Z 6 [Warning] [MY-010453] [Server] root@localhost is created with an empty password ! Please consider switching off the --initialize-insecure option.
lab_web_admin   | [Mon Jul 27 09:28:09 2026] [::ffff:172.18.0.1]:46802 Accepted
lab_web_admin   | [Mon Jul 27 09:28:09 2026] [::ffff:172.18.0.1]:46802 [200]: GET /?file=logo.png&version=5.5.0
lab_web_admin   | [Mon Jul 27 09:28:09 2026] [::ffff:172.18.0.1]:46802 Closing
lab_web_admin   | [Mon Jul 27 09:29:02 2026] [::ffff:172.18.0.1]:47682 Accepted
lab_web_admin   | [Mon Jul 27 09:29:02 2026] [::ffff:172.18.0.1]:47682 [302]: POST /
lab_web_admin   | [Mon Jul 27 09:29:02 2026] [::ffff:172.18.0.1]:47682 Closing
lab_web_admin   | [Mon Jul 27 09:29:02 2026] [::ffff:172.18.0.1]:47690 Accepted
lab_web_admin   | [Mon Jul 27 09:29:02 2026] [::ffff:172.18.0.1]:47690 [200]: GET /?server=db&username=root&db=lab_database
redis           | 1:M 27 Jul 2026 09:47:29.067 * <bf>     { cf-bucket-size      :         2 }
redis           | 1:M 27 Jul 2026 09:47:29.067 * <bf>     { cf-initial-size     :      1024 }
redis           | 1:M 27 Jul 2026 09:47:29.067 * <bf>     { cf-max-iterations   :        20 }
redis           | 1:M 27 Jul 2026 09:47:29.067 * <bf>     { cf-expansion-factor :         1 }
redis           | 1:M 27 Jul 2026 09:47:29.067 * <bf>     { cf-max-expansions   :        32 }
redis           | 1:M 27 Jul 2026 09:47:29.067 * <bf> ]
redis           | 1:M 27 Jul 2026 09:47:29.068 * Module 'bf' loaded from /usr/local/lib/redis/modules//redisbloom.so
redis           | 1:M 27 Jul 2026 09:47:29.084 * <search> search-workers default: 16 (min of MAX_WORKER_THREADS=16 and CPU cores)
redis           | 1:M 27 Jul 2026 09:47:29.085 * <search> Redis version found by RedisSearch : 8.8.0 - oss
redis           | 1:M 27 Jul 2026 09:47:29.085 * <search> RediSearch version 8.8.0 (Git=d2ae026)
redis           | 1:M 27 Jul 2026 09:47:29.086 * <search> Low level api version 1 initialized successfully
redis           | 1:M 27 Jul 2026 09:47:29.086 * <search> gc: ON, prefix min length: 2, min word length to stem: 4, prefix max expansions: 200, query timeout (ms): 500, timeout policy: return, oom policy: return, cursor read size: 1000, cursor max idle (ms): 300000, max doctable size: 1000000, max number of search results:  1000000, default scorer: BM25STD,
redis           | 1:M 27 Jul 2026 09:47:29.086 * <search> Initialized thread pools!
redis           | 1:M 27 Jul 2026 09:47:29.086 * <search> Enabled workers threadpool of size 16
redis           | 1:M 27 Jul 2026 09:47:29.092 * <search> Subscribe to config changes
redis           | 1:M 27 Jul 2026 09:47:29.092 * <search> Subscribe to cluster slot migration events
lab_mysql_db    | 2026-07-27 09:24:25+00:00 [Note] [Entrypoint]: Database files initialized
web             | /docker-entrypoint.sh: Launching /docker-entrypoint.d/20-envsubst-on-templates.sh
lab_web_admin   | [Mon Jul 27 09:29:02 2026] [::ffff:172.18.0.1]:47690 Closing
lab_web_admin   | [Mon Jul 27 09:29:02 2026] [::ffff:172.18.0.1]:47706 Accepted
lab_web_admin   | [Mon Jul 27 09:29:02 2026] [::ffff:172.18.0.1]:47706 [200]: GET /?file=jush.js&version=5.5.0
lab_web_admin   | [Mon Jul 27 09:29:02 2026] [::ffff:172.18.0.1]:47706 Closing
redis           | 1:M 27 Jul 2026 09:47:29.092 * <search> Enabled role change notification
redis           | 1:M 27 Jul 2026 09:47:29.092 * <search> Cluster configuration: AUTO partitions, type: 0, coordinator timeout: 0ms
redis           | 1:M 27 Jul 2026 09:47:29.093 * Module 'search' loaded from /usr/local/lib/redis/modules//redisearch.so
redis           | 1:M 27 Jul 2026 09:47:29.094 * <timeseries> RedisTimeSeries version 80800, git_sha=42ca4f1078fca732b7f9256adbf25914d67e1cc9
redis           | 1:M 27 Jul 2026 09:47:29.094 * <timeseries> Redis version found by RedisTimeSeries : 8.8.0 - oss
redis           | 1:M 27 Jul 2026 09:47:29.094 * <timeseries> Registering configuration options: [
redis           | 1:M 27 Jul 2026 09:47:29.094 * <timeseries>     { ts-compaction-policy   :              }
redis           | 1:M 27 Jul 2026 09:47:29.094 * <timeseries>     { ts-num-threads         :            3 }
redis           | 1:M 27 Jul 2026 09:47:29.094 * <timeseries>     { ts-libmr-protocol      :     INTERNAL }
redis           | 1:M 27 Jul 2026 09:47:29.094 * <timeseries>     { ts-retention-policy    :            0 }
redis           | 1:M 27 Jul 2026 09:47:29.094 * <timeseries>     { ts-duplicate-policy    :        block }
redis           | 1:M 27 Jul 2026 09:47:29.095 * <timeseries>     { ts-chunk-size-bytes    :         4096 }
redis           | 1:M 27 Jul 2026 09:47:29.095 * <timeseries>     { ts-encoding            :   compressed }
redis           | 1:M 27 Jul 2026 09:47:29.095 * <timeseries>     { ts-ignore-max-time-diff:            0 }
redis           | 1:M 27 Jul 2026 09:47:29.095 * <timeseries>     { ts-ignore-max-val-diff :     0.000000 }
redis           | 1:M 27 Jul 2026 09:47:29.095 * <timeseries> ]
redis           | 1:M 27 Jul 2026 09:47:29.095 * <timeseries> Detected redis oss
redis           | 1:M 27 Jul 2026 09:47:29.095 * <timeseries> Subscribe to ASM events
redis           | 1:M 27 Jul 2026 09:47:29.095 * <timeseries> Enabled diskless replication
```

Name: Darshika Singh
Roll No: 2401114
Batch: T22





Name: Darshika Singh
Roll No: 2401114
Batch: T22





Conclusion:

In this experiment, a multi-tier application stack comprising MySQL, Adminer, Nginx, and Redis was successfully orchestrated using Docker Compose. By defining services, port mappings, environment variables, and persistent storage in a single docker-compose.yml manifest, the process of configuring and managing complex multi-container environments was streamlined compared to running individual container commands. Interactive terminal verification confirmed service health across all mapped endpoints.

Mapping to Learning Outcome 4 (LO4)

This experiment successfully fulfills **LO4** (*Deploy, manage, and orchestrate multi-container application architectures using containerization tools*). By executing single-command stack

deployments (docker compose up), managing service-level isolation, establishing persistent named volumes for database integrity, and leveraging Docker's automated internal bridge networking for inter-service communication, practical competence in real-world DevOps container orchestration was effectively demonstrated.